

# 1. OpenClaw Deployment

---

===== [Note] =====

This section serves as a comprehensive installation and deployment tutorial for OpenClaw. As it involves numerous steps, if you wish to start using OpenClaw quickly, please:

Follow the instructions in the first subsection of this chapter: [1. Directly Use the Pre-configured Environment (Recommended)].

=====

## 1. OpenClaw Deployment

### 1.1 OpenClaw Installation

#### 1.1.1 Installing OpenClaw

#### 1.1.3 Accessing the OpenClaw Control UI on a PC (Windows)

### 1.2 Configuring Models (Must Read)

#### 1.2.1 Why Configure Models?

#### 1.2.2 How to Configure Models

#### 1. OpenRouter (OpenClaw's Built-in Service Provider)

#### 2. Bailian (Example: Third-Party Service Provider API)

This chapter covers two main topics:

- OpenClaw Installation (Jetson Nano Docker Scenario)
- Model Configuration (OpenRouter / BaiLian / Ollama)

---

## 1.1 OpenClaw Installation

---

OpenClaw Official Website: <https://openclaw.ai/>

Open-Source Project Repository: <https://github.com/openclaw/openclaw>

### 1.1.1 Installing OpenClaw

The Jetson Nano B01 requires a Docker environment for operation. OpenClaw comes pre-configured within the official Yabo Intelligent system image; therefore, the content in this section is provided solely for your reference should you wish to perform the installation manually.

The system version on the Jetson Nano B01 is relatively old and does not support Node.js versions higher than Node 22; consequently, installation via Docker is mandatory. ### 1.1.2 Installation using QuickStart

We will use the installation script officially provided by OpenClaw:

```
curl -fsSL https://openclaw.ai/install.sh | bash
```

```

root@jetson-yahboom:/# curl -fsSL https://openclaw.ai/install.sh | bash

🐾 OpenClaw Installer
Your messages, your servers, Meta's tears.

✓ Detected: linux

Install plan
OS: linux
Install method: npm
Requested version: latest

[1/3] Preparing environment
· Node.js not found, installing it now
· Installing Node.js via NodeSource
· Installing Linux build tools (make/g++/cmake/python3)
✓ Build tools installed
✓ Node.js v22 installed
· Active Node.js: v22.22.1 (/usr/bin/node)
· Active npm: 10.9.4 (/usr/bin/npm)

[2/3] Installing OpenClaw
✓ Git already installed
· Installing OpenClaw v2026.3.7
✓ OpenClaw npm package installed
✓ OpenClaw installed

[3/3] Finalizing setup

🐾 OpenClaw installed successfully (2026.3.7)!
Settled in. Time to automate your life whether you're ready or not.

· Starting setup

🐾 OpenClaw 2026.3.7 (42a1394) – I run on caffeine, JSON5, and the audacity of "it worked on my machine."

```

During this process, the installation script will automatically detect any missing environment dependencies and assist us in installing them with a single click.

During the installation process, use the arrow keys (Up, Down, Left, Right) on your keyboard to navigate options; press the Spacebar to select/deselect an item, and press Enter to confirm and proceed with configuration.

```

🐾 OpenClaw 2026.3.7 (42a1394) – I run on caffeine, JSON5, and the audacity of "it worked on my machine."

```

# OPENCLAW

🐾 OPENCLAW 🐾

OpenClaw onboarding

◉ Security

Security warning – please read.

OpenClaw is a hobby project and still in beta. Expect sharp edges.  
By default, OpenClaw is a personal agent: one trusted operator boundary.  
This bot can read files and run actions if tools are enabled.  
A bad prompt can trick it into doing unsafe things.

OpenClaw is not a hostile multi-tenant boundary by default.  
If multiple users can message one tool-enabled agent, they share that delegated tool authority.

If you're not comfortable with security hardening and access control, don't run OpenClaw.  
Ask someone experienced to help before enabling tools or exposing it to the internet.

Recommended baseline:

- Pairing/allowlists + mention gating.
- Multi-user/shared inbox: split trust boundaries (separate gateway/credentials, ideally separate OS users/hosts).
- Sandbox + least-privilege tools.
- Shared inboxes: isolate DM sessions (`session.dmScope: per-channel-peer`) and keep

```
A bad prompt can trick it into doing unsafe things.

OpenClaw is not a hostile multi-tenant boundary by default.
If multiple users can message one tool-enabled agent, they share that delegated tool
authority.

If you're not comfortable with security hardening and access control, don't run
OpenClaw.
Ask someone experienced to help before enabling tools or exposing it to the internet.

Recommended baseline:
- Pairing/allowlists + mention gating.
- Multi-user/shared inbox: split trust boundaries (separate gateway/credentials, ideally
  separate OS users/hosts).
- Sandbox + least-privilege tools.
- Shared inboxes: isolate DM sessions (`session.dmScope: per-channel-peer`) and keep
  tool access minimal.
- Keep secrets out of the agent's reachable filesystem.
- Use the strongest available model for any bot with tools or untrusted inboxes.

Run regularly:
openclaw security audit --deep
openclaw security audit --fix

Must read: https://docs.openclaw.ai/gateway/security
```

- ◊ I understand this is personal-by-default and shared/multi-user use requires lock-down. Continue?  
Yes
- ◆ Onboarding mode
  - QuickStart (Configure details later via openclaw configure.)
  - Manual

Here, we will select the "QuickStart" option to rapidly complete the OpenClaw installation;  
we can defer the specific configuration details to a later stage.

If you have not yet applied for any API Keys, simply select "Skip" for these steps for now; you can  
configure them later.

```
Tailscale exposure: Off
Direct to chat channels.
```

- ◆ Model/auth provider
  - OpenAI
  - Anthropic
  - Chutes
  - vLLM
  - MiniMax
  - Moonshot AI (Kimi K2.5)
  - Google
  - xAI (Grok)
  - Mistral AI
  - Volcano Engine
  - BytePlus
  - OpenRouter
  - Kilo Gateway
  - Qwen
  - Z.AI
  - Qianfan
  - Copilot
  - Vercel AI Gateway
  - OpenCode Zen
  - Xiaomi
  - Synthetic
  - Together AI
  - Hugging Face
  - Venice AI
  - LiteLLM
  - Cloudflare AI Gateway
  - Custom Provider
  - skip for now

Tailscale exposure: Off  
Direct to chat channels.

- ◊ Model/auth provider
  - Skip for now
- ◆ Filter models by provider
  - All providers
    - amazon-bedrock
    - anthropic
    - azure-openai-responses
    - cerebras
    - github-copilot
    - google
    - google-antigravity
    - google-gemini-cli
    - google-vertex
    - groq
    - huggingface
    - kimi-coding
    - minimax
    - minimax-cn
    - mistral
    - ollama
    - openai
    - openai-codex
    - opencode
    - openrouter
    - vercel-ai-gateway
    - xai
    - zai

For the model selection, we will choose to keep the default settings (to be configured later).

All providers

- ◆ Default model
  - Keep current (default: anthropic/claude-opus-4-6)
    - Enter model manually
    - amazon-bedrock/anthropic.claude-3-haiku-20240307-v1:0
    - amazon-bedrock/anthropic.claude-3-5-haiku-20241022-v1:0
    - amazon-bedrock/anthropic.claude-haiku-4-5-20251001-v1:0
    - amazon-bedrock/eu.anthropic.claude-haiku-4-5-20251001-v1:0
    - amazon-bedrock/global.anthropic.claude-haiku-4-5-20251001-v1:0
    - amazon-bedrock/us.anthropic.claude-haiku-4-5-20251001-v1:0
    - amazon-bedrock/anthropic.claude-3-opus-20240229-v1:0
    - amazon-bedrock/anthropic.claude-opus-4-20250514-v1:0
    - amazon-bedrock/us.anthropic.claude-opus-4-20250514-v1:0
    - amazon-bedrock/anthropic.claude-opus-4-1-20250805-v1:0
    - amazon-bedrock/us.anthropic.claude-opus-4-1-20250805-v1:0
    - amazon-bedrock/anthropic.claude-opus-4-5-20251101-v1:0
    - amazon-bedrock/eu.anthropic.claude-opus-4-5-20251101-v1:0
    - amazon-bedrock/global.anthropic.claude-opus-4-5-20251101-v1:0
    - amazon-bedrock/us.anthropic.claude-opus-4-5-20251101-v1:0
    - amazon-bedrock/anthropic.claude-opus-4-6-v1
    - amazon-bedrock/eu.anthropic.claude-opus-4-6-v1
    - amazon-bedrock/global.anthropic.claude-opus-4-6-v1
    - amazon-bedrock/us.anthropic.claude-opus-4-6-v1
    - amazon-bedrock/anthropic.claude-3-sonnet-20240229-v1:0
    - amazon-bedrock/anthropic.claude-3-5-sonnet-20240620-v1:0
    - amazon-bedrock/anthropic.claude-3-5-sonnet-20241022-v2:0
    - amazon-bedrock/anthropic.claude-3-7-sonnet-20250219-v1:0
    - amazon-bedrock/anthropic.claude-sonnet-4-20250514-v1:0
    - amazon-bedrock/eu.anthropic.claude-sonnet-4-20250514-v1:0
    - amazon-bedrock/global.anthropic.claude-sonnet-4-20250514-v1:0
    - amazon-bedrock/us.anthropic.claude-sonnet-4-20250514-v1:0
    - amazon-bedrock/anthropic.claude-sonnet-4-5-20250929-v1:0
    - ...

Regarding the "Channel" selection—which is primarily used for communication with third-party applications—we will also choose to skip it for the time being.

```
Microsoft Teams: Bot Framework; enterprise support.
Mattermost: self-hosted Slack-style chat; install the plugin to enable.
Nextcloud Talk: Self-hosted chat via Nextcloud Talk webhook bots.
Matrix: open protocol; install the plugin to enable.
BlueBubbles: iMessage via the BlueBubbles mac app + REST API.
Zalo: Vietnam-focused messaging platform with Bot API.
Zalo Personal: Zalo personal account via QR code login.
Synology Chat: Connect your Synology NAS Chat to OpenClaw with full agent capabilities.
Tlon: decentralized messaging on Urbit; install the plugin to enable.
```

- ◆ **Select channel (QuickStart)**
  - Telegram (Bot API)
  - WhatsApp (QR link)
  - Discord (Bot API)
  - IRC (Server + Nick)
  - Google Chat (Chat API)
  - Slack (Socket Mode)
  - Signal (signal-cli)
  - iMessage (img)
  - LINE (Messaging API)
  - Feishu/Lark (飞书)
  - Nostr (NIP-04 DMs)
  - Microsoft Teams (Bot Framework)
  - Mattermost (plugin)
  - Nextcloud Talk (self-hosted)
  - Matrix (plugin)
  - BlueBubbles (macOS app)
  - Zalo (Bot API)
  - Zalo (Personal Account)
  - Synology Chat (Webhook)
  - Tlon (Urbit)
- **Skip for now** (You can add channels later via `openclaw channels add`)

Search Provider refers to the API provider for search services; we choose to skip this step.

```
Microsoft Teams: Bot Framework; enterprise support.
Mattermost: self-hosted Slack-style chat; install the plugin to enable.
Nextcloud Talk: Self-hosted chat via Nextcloud Talk webhook bots.
Matrix: open protocol; install the plugin to enable.
BlueBubbles: iMessage via the BlueBubbles mac app + REST API.
Zalo: Vietnam-focused messaging platform with Bot API.
Zalo Personal: Zalo personal account via QR code login.
Synology Chat: Connect your Synology NAS Chat to OpenClaw with full agent capabilities.
Tlon: decentralized messaging on Urbit; install the plugin to enable.
```

- ◆ **Select channel (QuickStart)**
    - Skip for now
- Updated ~/.openclaw/openclaw.json  
Workspace OK: ~/.openclaw/workspace  
Sessions OK: ~/.openclaw/agents/main/sessions
- ◆ **Web search**
    - Web search lets your agent look things up online.  
Choose a provider and paste your API key.  
Docs: <https://docs.openclaw.ai/tools/web>
- ◆ **Search provider**
    - Perplexity Search
    - Brave Search
    - Gemini (Google Search)
    - Grok (xAI)
    - Kimi (Moonshot)
  - **Skip for now** (Configure later with `openclaw configure --section web`)

For the "Skills" configuration, we choose not to configure them for now (select "no").

```
Zalo Personal: Zalo personal account via QR code login.
Synology Chat: Connect your Synology NAS Chat to OpenClaw with full agent capabilities.
Tlon: decentralized messaging on Orbit; install the plugin to enable.

◇ Select channel (QuickStart)
| Skip for now
Updated ~/.openclaw/openclaw.json
Workspace OK: ~/.openclaw/workspace
Sessions OK: ~/.openclaw/agents/main/sessions

◇ Web search
Web search lets your agent look things up online.
Choose a provider and paste your API key.
Docs: https://docs.openclaw.ai/tools/web

◇ Search provider
Skip for now

◇ Skills status
Eligible: 4
Missing requirements: 40
Unsupported on this OS: 7
Blocked by allowlist: 0

◆ Configure skills now? (recommended)
○ Yes / ● No
```

For the "Hooks" configuration, we press the Spacebar to select "Skip for now," and then press Enter to confirm.

```
Docs: https://docs.openclaw.ai/tools/web

◇ Search provider
Skip for now

◇ Skills status
Eligible: 4
Missing requirements: 40
Unsupported on this OS: 7
Blocked by allowlist: 0

◇ Configure skills now? (recommended)
No

◇ Hooks
Hooks let you automate actions when agent commands are issued.
Example: Save session context to memory when you issue /new or /reset.
Learn more: https://docs.openclaw.ai/automation/hooks

◆ Enable hooks?
■ Skip for now
□ boot-md
□ bootstrap-extra-files
□ command-logger
□ session-memory
```

Wait for the installation to complete.

Once the installation is finished, we copy this URL for future reference. (Note: In subsequent tutorials, you may observe that the token changes; this is because we later manually configured the Gateway. Therefore, it is not strictly necessary to record this specific URL here, as it will change later anyway.)

```
|
├─ Security ──
│  Running agents on your computer is risky – harden your setup:
│  https://docs.openclaw.ai/security
│
├─ Shell completion ──
│  Shell completion installed. Restart your shell or run: source ~/.zshrc
│
├─ Dashboard ready ──
│  Dashboard link (with token):
│  http://127.0.0.1:18789/#token=b45c9f0d90339c275a2da1db7735e597bfda200e3900b7e1
│  Copy/paste this URL in a browser on this machine to control OpenClaw.
│  No GUI detected. Open from your computer:
│  ssh -N -L 18789:127.0.0.1:18789 user@host>
│  Then open:
│  http://localhost:18789/
│  http://localhost:18789/#token=b45c9f0d90339c275a2da1db7735e597bfda200e3900b7e1
│  docs:
│  https://docs.openclaw.ai/gateway/remote
│  https://docs.openclaw.ai/web/control-ui
│
├─ Web search ──
│  Web search was skipped. You can enable it later:
│  openclaw configure --section web
│  Docs: https://docs.openclaw.ai/tools/web
│
├─ What now ──
│  What now: https://openclaw.ai/showcase ("What People Are Building").
│
└─ Onboarding complete. Use the dashboard link above to control OpenClaw.

root@jetson-yahboom:/#
```

### 1.1.3 Accessing the OpenClaw Control UI on a PC (Windows)

**Note on Common Issues:** Please use a recent version of the Edge or Chrome browser; other browsers may encounter a "4008" error.

After deploying OpenClaw version 3.8 within a Docker container on a Jetson Nano, attempting to access the OpenClaw Control UI (i.e., the web console) directly via a browser on the Jetson device itself will result in a "4008" access error. Therefore, you must access the interface using a browser on a separate computer.

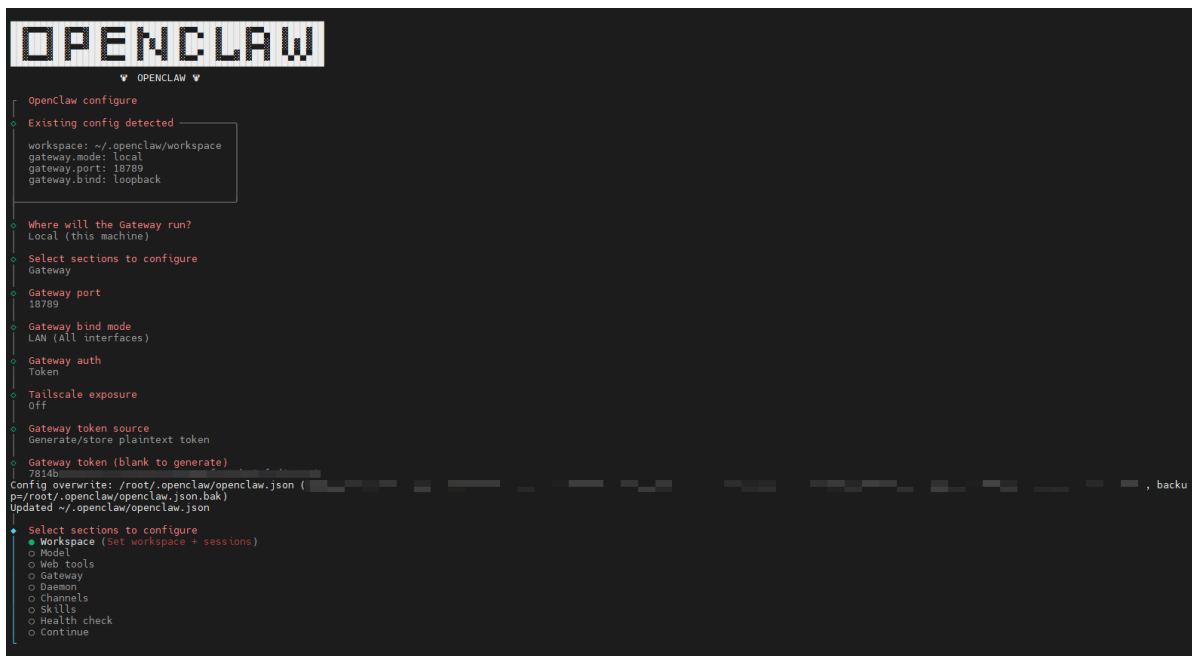
If no other computer is available, you can utilize OpenClaw's TUI tool, which operates via the command line within a terminal. For specific instructions, please refer to the "Using the TUI" section later in this document.

1. Modify the OpenClaw configuration file to allow other devices within your local network to access OpenClaw.

```
openclaw config
```

At this step, please make your selections as shown in the image below (use the Up/Down arrow keys to navigate, and press Enter to confirm).

Note: The "Token" value may differ from what is shown in our image, but all other settings must match exactly. The critical setting here is the "Gateway bind mode" option; you must set this to "LAN" to enable other devices within your local area network to access your OpenClaw instance.



```
OPENCRAW
▼ OPENCRAW ▼

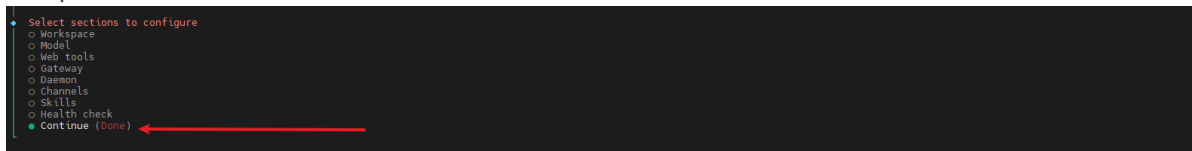
OpenClaw configure
Existing config detected
workspace: ~/.opencrawl/workspace
gateway.mode: local
gateway.port: 18789
gateway.bind: loopback

Where will the Gateway run?
Local (this machine)

Select sections to configure
Gateway
Gateway port
18789
Gateway bind mode
LAN (All interfaces)
Gateway auth
Token
Tailscale exposure
Off
Gateway token source
Generate/store plaintext token
Gateway token (blank to generate)
7814b
Config overwrite: /root/.opencrawl/opencrawl.json (
p=/root/.opencrawl/opencrawl.json.bak)
Updated ~/.opencrawl/opencrawl.json

Select sections to configure
• Workspace (Set workspace + sessions)
  • Model
  • Web tools
  • Gateway
  • Daemon
  • Channels
  • Skills
  • Health check
  • Continue
```

Finally, select "Continue" to confirm your choices, press Enter, and the configuration process is complete!



```
Select sections to configure
  • Workspace (Set workspace + sessions)
    • Model
    • Web tools
    • Gateway
    • Daemon
    • Channels
    • Skills
    • Health check
    • Continue (Done) ←
```

Once the configuration is complete, let's first start the OpenClaw Gateway. Open a new terminal window and enter the following command:

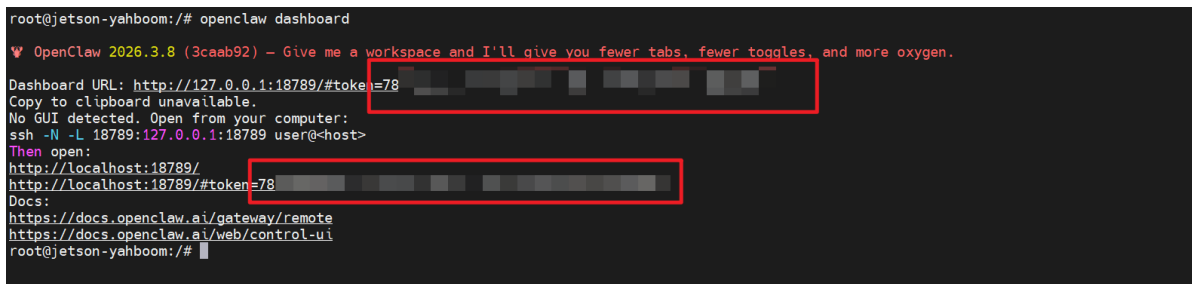
```
opencrawl gateway
```

## 2. Obtain the Access Token

The Token referred to here is the one we generated during the configuration process. If you didn't write it down, you can use the following command:

```
opencrawl dashboard
```

This command will display the token for you once. Alternatively, you can locate the `~/.opencrawl/opencrawl.json` file; your token will be stored within this configuration file.

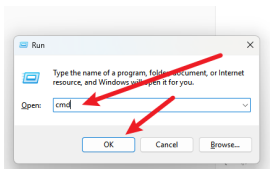


```
root@jetson-yahboom:/# opencrawl dashboard
▼ OpenClaw 2026.3.8 (3caab92) - Give me a workspace and I'll give you fewer tabs, fewer toggles, and more oxygen.
Dashboard URL: http://127.0.0.1:18789/#token=78
Copy to clipboard unavailable.
No GUI detected. Open from your computer:
ssh -N -L 18789:127.0.0.1:18789 user@<host>
Then open:
http://localhost:18789/
http://localhost:18789/#token=78
Docs:
https://docs.opencrawl.ai/gateway/remote
https://docs.opencrawl.ai/web/control-ui
root@jetson-yahboom:/#
```

## 3. Configure the Connection Environment to the Jetson on a PC within the Same Local Network

Open a Command Prompt (Cmd) terminal on your PC. (Using Windows as an example: press `win + R` to open the Run dialog, type `cmd`, and then press Enter.)





Enter the following command into the console:

```
ssh -L 18789:127.0.0.1:18789 jetson@<jetson-ip-address>
```

Note: You must replace `<jetson-ip-address>` with the actual IP address of your Jetson board.

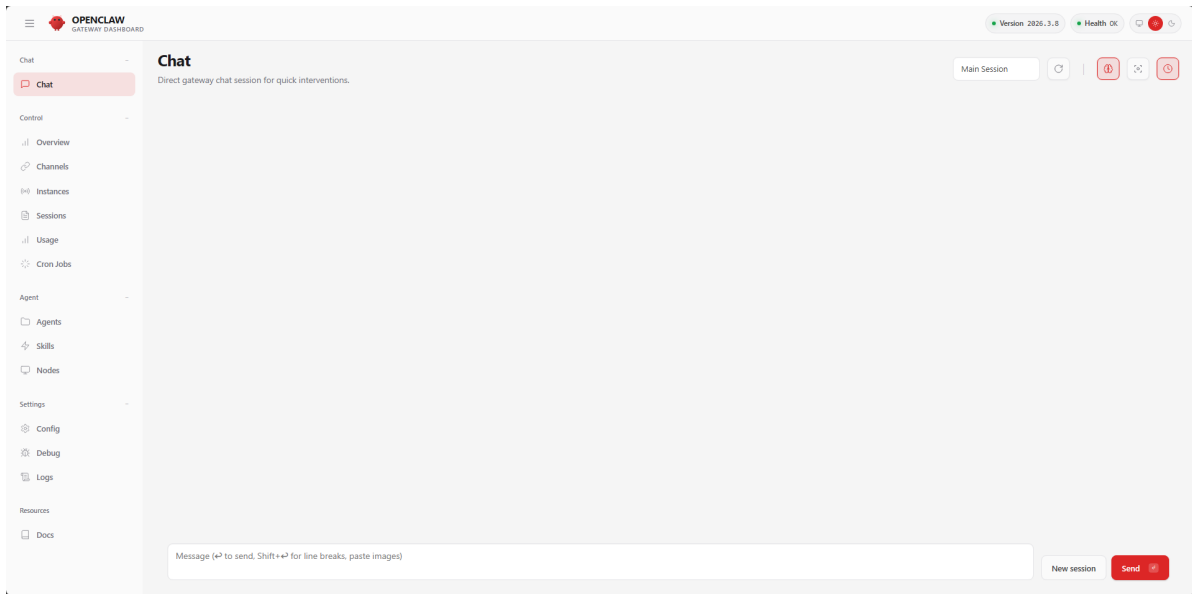
After pressing Enter, you will be prompted to enter the password (the default password is `yahboom`).

```
jetson@jetson-yahboom: ~  
Microsoft Windows [Version 10.0.26200.7840]  
(c) Microsoft Corporation. All rights reserved.  
C:\Users\Administrator>ssh -L 18789:127.0.0.1:18789 jetson@192.168.2.56  
jetson@192.168.2.56's password:  
Welcome to Ubuntu 18.04.6 LTS (GNU/Linux 4.9.299-tegra aarch64)  
  
* Documentation:  https://help.ubuntu.com  
* Management:    https://landscape.canonical.com  
* Support:        https://ubuntu.com/advantage  
This system has been minimized by removing packages and content that are  
not required on a system that users do not log into.  
  
To restore this content, you can run the 'unminimize' command.  
  
219 updates can be applied immediately.  
166 of these updates are standard security updates.  
To see these additional updates run: apt list --upgradable  
  
Last login: Tue Mar 10 12:27:26 2026 from 192.168.2.42  
jetson@jetson-yahboom:~$
```

If this screen appears, it indicates that you have successfully connected to the Jetson.

#### 4. Access via Web Browser

Open a web browser on your PC and enter the full address—consisting of the IP address combined with the token—into the address bar (this is the address we obtained using the `openc1aw` command earlier). Once you see this interface at the address output by the dashboard (e.g., `https://127.0.0.1:18789/#token=xxxxxxxxxxxxxx`), it means the deployment was successful!



## 1.2 Configuring Models (Must Read)

### 1.2.1 Why Configure Models?

OpenClaw serves as an AI Agent gateway; on its own, it does not possess the "intelligent" capabilities of a large language model. We must configure a large model to endow OpenClaw with the ability to think and comprehend.

### 1.2.2 How to Configure Models

Here, we will use three representative models as examples:

1. OpenRouter (An OpenClaw-integrated provider, accessible both domestically and internationally)
2. Alibaba Bailian Large Model Platform (A third-party provider; requires manual configuration)
3. Ollama (A local large model)

Before you begin the configuration process, you must first start the OpenClaw Gateway. > If you did not exit the Gateway terminal after completing the deployment in the previous chapter, you may skip this step.

Enter the following command in your terminal:

```
openclaw gateway
```

```
root@jetson-yahboom:~/# openclaw gateway
▼ OpenClaw 2026.3.8 (3caab92) - Built by lobsters, for humans. Don't question the hierarchy.
14:54:59 Config overwrite: /root/.openclaw/openclaw.json (sha256 c0020f6c3fe15773946517f6949c8badcb892db7e1482c0c0f7736be8e3e9e60 -> 46b602c1773a9e88f0b137cdbcdb6696097642bd99b7bc2d503af8622fb4704, backups/root/.openclaw/openclaw.json.bak)
14:54:59 [gateway] seeded gateway.controlui.allowedOrigins ["http://localhost:18789","http://127.0.0.1:18789"] for bind-lan (required since v2026.2.26; see issue #29385). Add other origins to gateway.controlui.allowedOrigins if needed.
14:55:00 [canvas] host mounted at http://0.0.0.0:18789/ openclaw_/canvas/ (/root/.root/.openclaw/canvas)
14:55:00 [gateway] ▲ Gateway is binding to a non-loopback address. Ensure authentication is configured before exposing to public networks.
14:55:00 [heartbeat] started
14:55:00 [health-monitor] started (interval: 300s, startup-grace: 60s, channel-connect-grace: 120s)
14:55:00 [gateway] agent model: anthropic/claude-opus-4-6
14:55:00 [gateway] listening on ws://0.0.0.0:18789 (PID 1069)
14:55:00 [gateway] log file: /tmp/openclaw/openclaw-2026-03-10.log
14:55:00 [browser/server] Browser control listening on http://127.0.0.1:18791/ (auth=token)
15:05:34 [gateway] device pairing auto-approved device=11f1bcd22481a3a2099f9d9c3f7f5cb915ce3609a9e9b70eb926fcfb7fa2adb role=operator
15:05:34 [ws] webchat connected cmd=aa9e05c-0577-4ee6-9e84-212d42f1bbe2 remote=127.0.0.1 client=openclaw-control-ui webchat v2026.3.8
15:05:35 [ws] a res / chat-history 0d4e cmd=aa9e05c-bbe2-11e61cc39a-48f5
```

# 1. OpenRouter (OpenClaw's Built-in Service Provider)

For this section, you must have already applied for an OpenRouter API Key by following the instructions in the previous tutorial (please refer to the earlier guide: [Online AI model course / 4. OpenRouter Large Model API Aggregation Platform]).

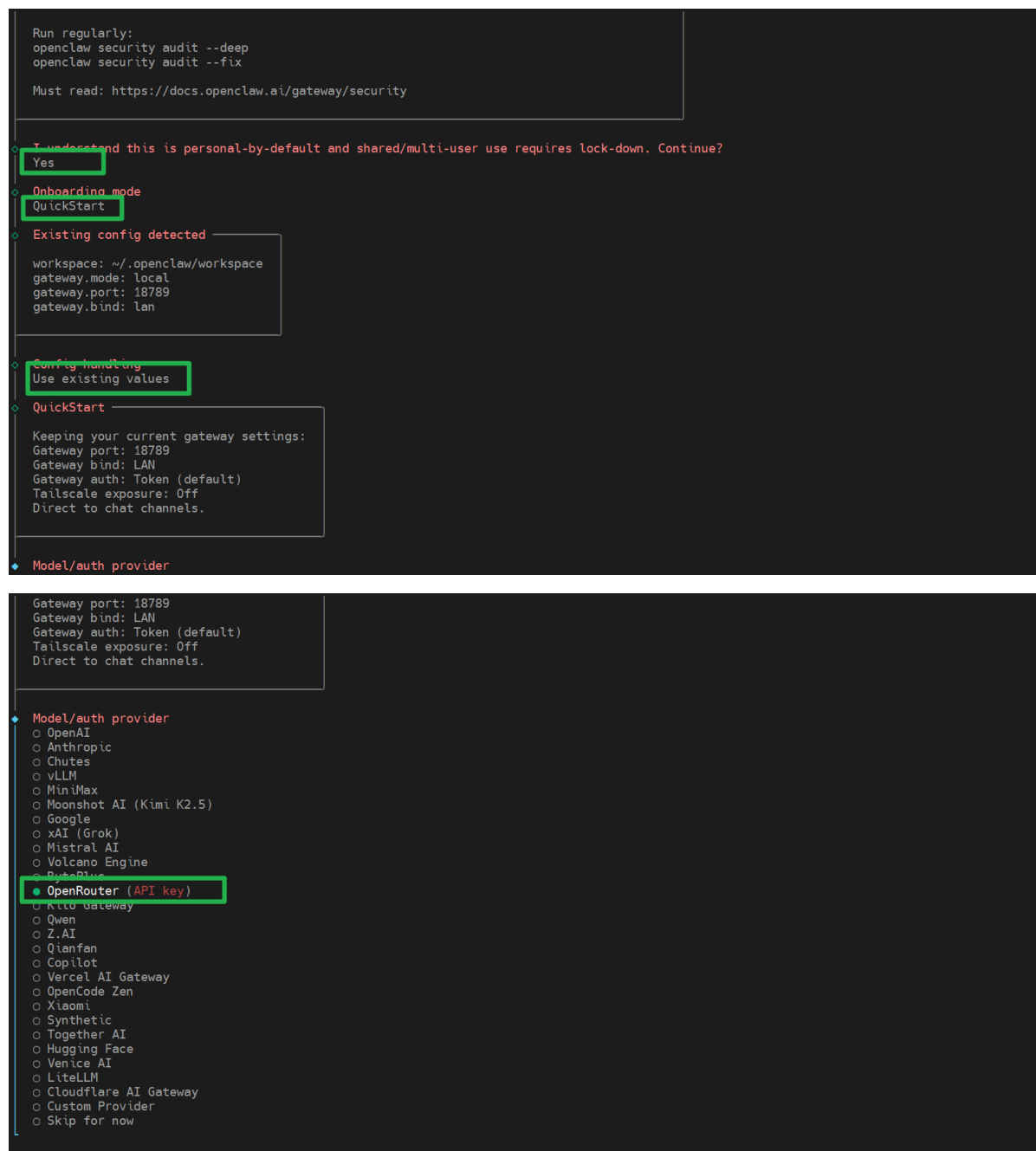
The format of an OpenRouter API Key typically looks like this: `sk-or-v1-`

`b3xxxxxx4xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxcddf81d9`

We will once again use the initial "onboard" wizard to configure the settings.

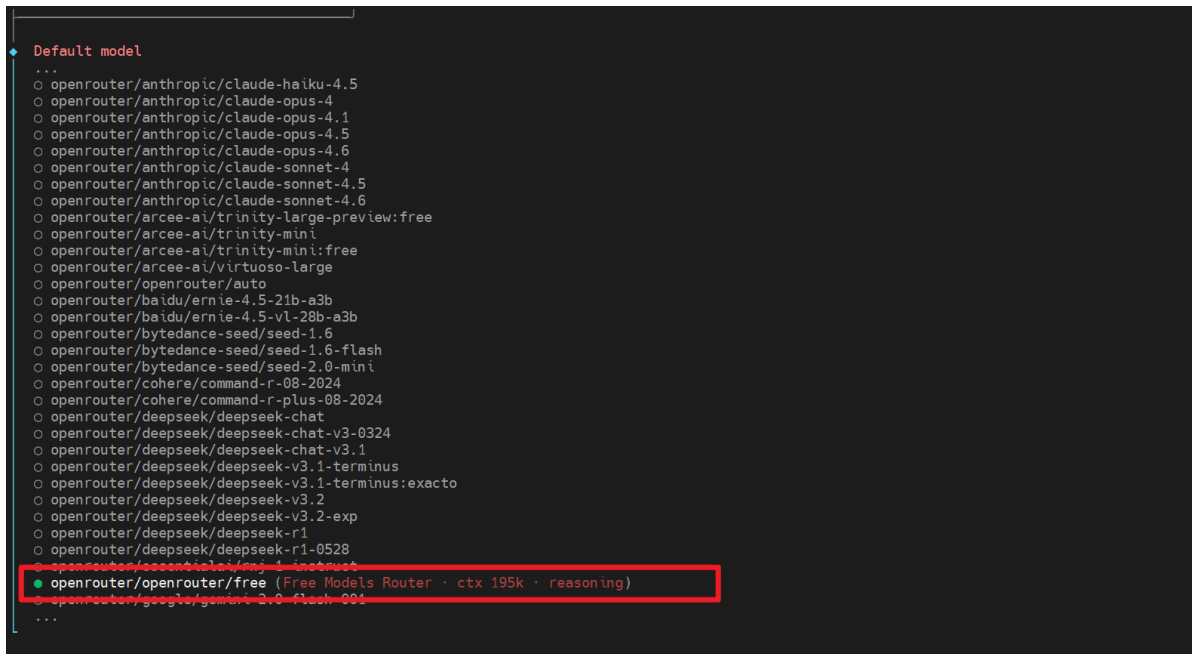
```
openc1aw onboard
```

Please select the options exactly as shown in the images below.



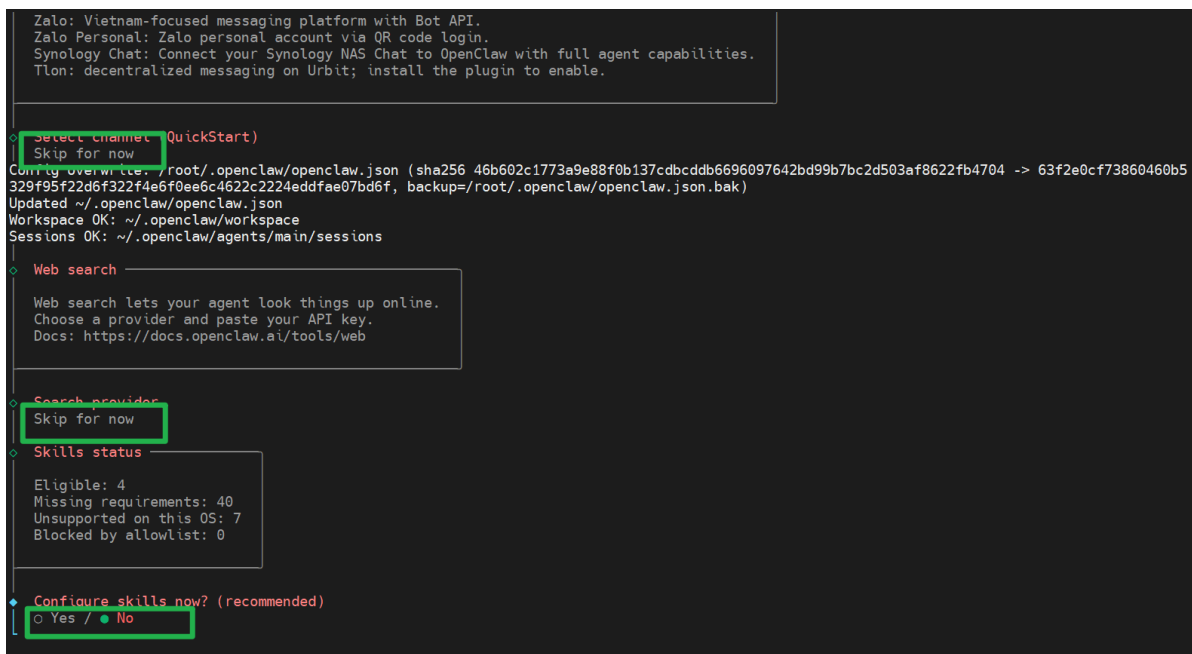
In the next step, we need to select a model. Due to OpenRouter's traffic control policies, we recommend maintaining an account balance of at least \$11. If your balance falls below this threshold, OpenRouter may throttle your request speeds (please note that this policy is subject to change; always refer to the latest actual conditions). Here, we have the option to select a free

model (identified by model names ending in "free"), or to choose a dedicated free model provided by OpenRouter: `openrouter/free`. (This acts as a router that randomly selects a free model from those currently available on OpenRouter. It intelligently filters for models that support the specific capabilities required by your request.)



At this stage, you may be prompted to enter an API Key. Simply paste your API Key into the field and press Enter to proceed to the next step.

Next, you will encounter various other configuration options. For now, we will skip these sections; select "Skip" for all of them.



```
Running agents on your computer is risky - harden your setup:
https://docs.openclaw.ai/security

Dashboard ready

Dashboard link (with token):
http://192.168.2.56:18789/#token=7814bc0c9670673701379ea20e0296fe53eb91cf6d18a3a4
Copy/paste this URL in a browser on this machine to control OpenClaw.
No GUI detected. Open from your computer:
ssh -N -L 18789:127.0.0.1:18789 user@<host>
Then open:
http://localhost:18789/
http://localhost:18789/#token=7814bc0c9670673701379ea20e0296fe53eb91cf6d18a3a4
Docs:
https://docs.openclaw.ai/gateway/remote
https://docs.openclaw.ai/web/control-ui

Web search

Web search was skipped. You can enable it later:
openclaw configure --section web
Docs: https://docs.openclaw.ai/tools/web

What now

What now: https://openclaw.ai/showcase ("What People Are Building").

Onboarding complete. Use the dashboard link above to control OpenClaw.
root@jetson-yahboom:/#
```

At this point, the configuration process is complete.

We can now use the TUI feature to perform a quick test.

Enter the following command:

```
openclaw tui
```

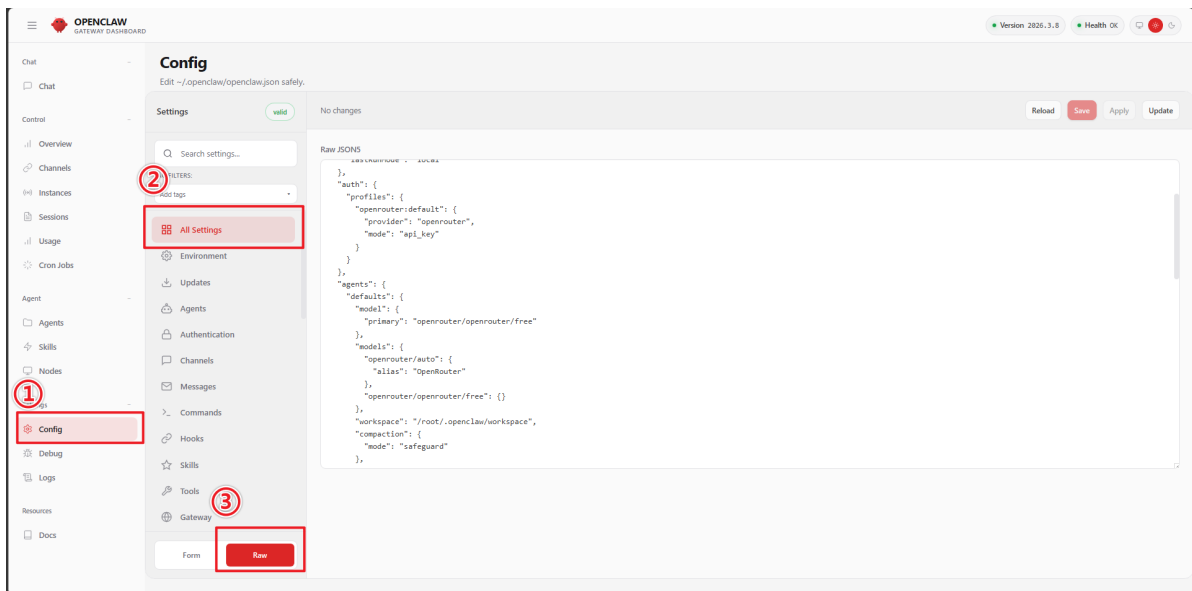
Once the command is entered, simply type a greeting to OpenClaw. (In the image below, the "Input" section displays the text you entered, while the green box highlights the model you selected.)

```
root@jetson-yahboom:/# openclaw tui
OpenClaw 2026.3.8 (3caab92) - Powered by open source, sustained by spite and good documentation.
openclaw tui - ws://127.0.0.1:18789 - agent main - session main
session agent:main:main
hello? ← Input
Hello! I'm here.
I just came online. Who am I? Who are you?
connected | idle
agent main | session main (openclaw-tui) | openrouter/openrouter/free | think low | tokens ?/200k
```

## 2. Bailian (Example: Third-Party Service Provider API)

Configuring a third-party API requires editing OpenClaw's configuration file (located at `~/ .openclaw/openclaw.json`). However, I highly recommend performing these edits via the web interface (the OpenClaw Control UI) instead.

Open the OpenClaw configuration page in your web browser, then click the following three buttons in sequence:



The current contents of the configuration file are displayed below; pay particular attention to the `agents` section.

```
{
  "meta": {
    "lastTouchedVersion": "2026.3.8",
    "lastTouchedAt": "2026-03-10T07:56:18.292z"
  },
  "wizard": {
    "lastRunAt": "2026-03-10T07:56:18.270z",
    "lastRunVersion": "2026.3.8",
    "lastRunCommand": "onboard",
    "lastRunMode": "local"
  },
  "auth": {
    "profiles": {
      "openrouter:default": {
        "provider": "openrouter",
        "mode": "api_key"
      }
    }
  },
  "agents": {
    "defaults": {
      "model": {
        "primary": "openrouter/openrouter/free"
      }
    },
    "models": {
      "openrouter/auto": {
        "alias": "OpenRouter"
      },
      "openrouter/openrouter/free": {}
    }
  },
  "workspace": "/root/.openclaw/workspace",
  "compaction": {
    "mode": "safeguard"
  },
  "maxConcurrent": 4,
  "subagents": {
    "maxConcurrent": 8
  }
}
```

```

    }
  },
  "tools": {
    "profile": "coding"
  },
  "messages": {
    "ackReactionScope": "group-mentions"
  },
  "commands": {
    "native": "auto",
    "nativeskills": "auto",
    "restart": true,
    "ownerDisplay": "raw"
  },
  "session": {
    "dmScope": "per-channel-peer"
  },
  "gateway": {
    "port": 18789,
    "mode": "local",
    "bind": "lan",
    "controlUi": {
      "allowedOrigins": [
        "http://localhost:18789",
        "http://127.0.0.1:18789"
      ]
    }
  },
  "auth": {
    "mode": "token",
    "token": "__OPENCLAW_REDACTED__"
  },
  "tailscale": {
    "mode": "off",
    "resetOnExit": false
  },
  "nodes": {
    "denyCommands": [
      "camera.snap",
      "camera.clip",
      "screen.record",
      "contacts.add",
      "calendar.add",
      "reminders.add",
      "sms.send"
    ]
  }
}

```

We need to add a "models" section (located at the top level of the JSON structure—meaning this newly added "models" object sits at the same hierarchical level as "agents").

Please note: here, you must replace `DASHSCOPE_API_KEY` with your actual Bailian API Key (refer to the previous section for instructions on how to obtain one).

For the "models" section here, we have used `qwen3.5-plus` and `qwen3-coder-next` as examples. If you wish to support other models, please consult the model list available at: <https://help.aliyun.com/zh/model-studio/models>

3. Replace the `"agents": {...}` section with the configuration below. Replace `DASHSCOPE_API_KEY` with your actual API key.

**Important** This configuration is for pay-as-you-go billing. If you use the Coding Plan, see [Coding Plan for OpenClaw](#) for plan-specific settings.

This example uses `qwen3.5-plus` and `qwen3-coder-next` in Singapore. The base URL, API key, and models must belong to the same region. To add other models, define them in `providers.baillian.models` (set `reasoning` to `false`) and add a `"baillian/<Model ID>": {}` entry to `agents.defaults.models`. See [Model list](#) for supported models per region.

```
"models": {
  "mode": "merge",
  "providers": {
    "baillian": {
      "baseUrl": "https://dashscope.aliyuncs.com/compatible-mode/v1",
      "apiKey": "DASHSCOPE_API_KEY",
      "api": "openai-completions",
      "models": [
        {
          "id": "qwen3.5-plus",
          "name": "qwen3.5-plus",
          "reasoning": false,
          "input": ["text", "image"],
          "contextwindow": 1000000,
          "maxTokens": 65536
        },
        {
          "id": "qwen3-coder-next",
          "name": "qwen3-coder-next",
          "reasoning": false,
          "input": ["text"],
          "contextwindow": 262144,
          "maxTokens": 65536
        }
      ]
    }
  }
},
```

After adding the "models," we also need to modify the "agents" section to set the BaiLian large language model as the default.

The "agents" configuration before modification is shown below (by default, it uses the OpenRouter service we configured in the previous step).

```
"agents": {
  "defaults": {
    "model": {
      "primary": "openrouter/openrouter/free"
    },
    "models": {
      "openrouter/auto": {
        "alias": "OpenRouter"
      },
      "openrouter/openrouter/free": {}
    },
    "workspace": "/root/.openclaw/workspace",
```



```

    "compaction": {
      "mode": "safeguard"
    },
    "maxConcurrent": 4,
    "subagents": {
      "maxConcurrent": 8
    }
  }
},

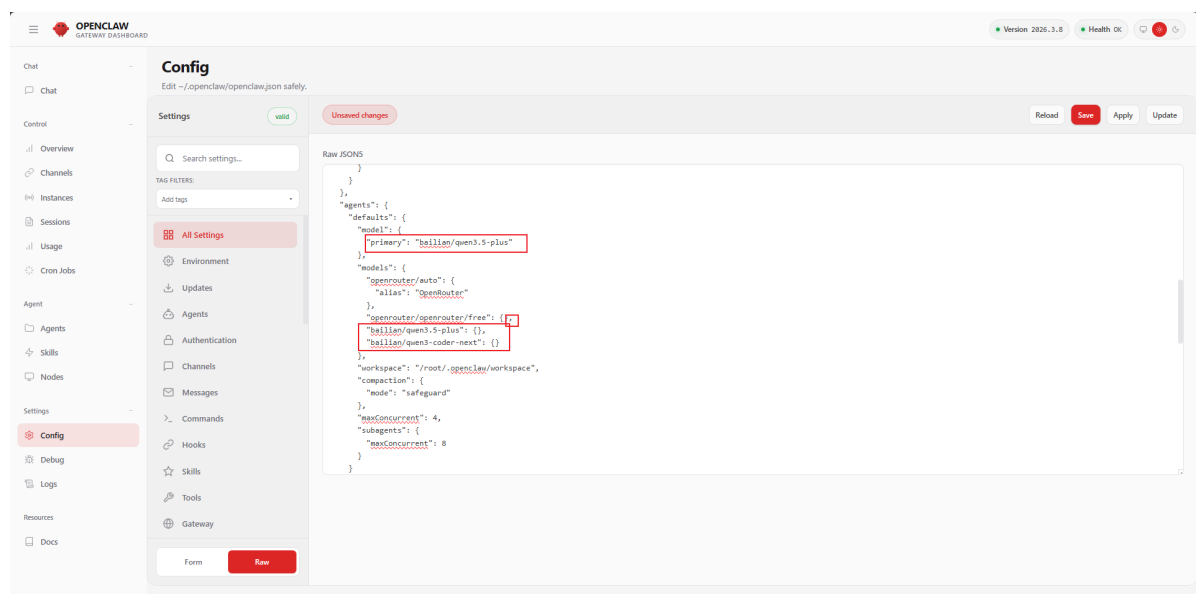
```

The modified agents are as follows:

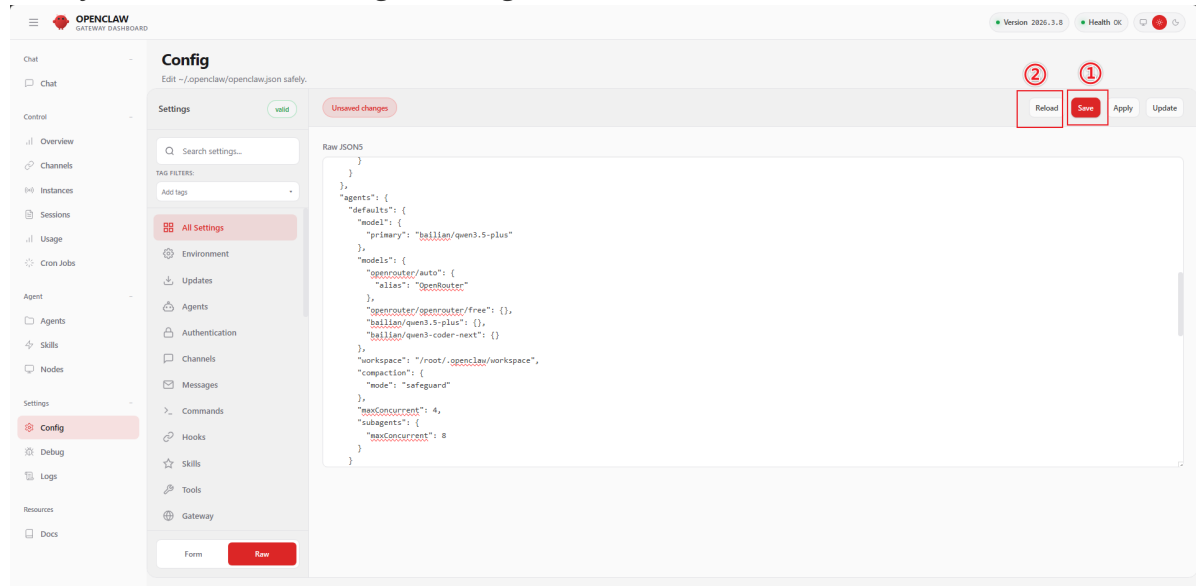
```

"agents": {
  "defaults": {
    "model": {
      "primary": "bailian/qwen3.5-plus"
    },
    "models": {
      "openrouter/auto": {
        "alias": "OpenRouter"
      },
      "openrouter/openrouter/free": {},
      "bailian/qwen3.5-plus": {},
      "bailian/qwen3-coder-next": {}
    },
    "workspace": "/root/.openclaw/workspace",
    "compaction": {
      "mode": "safeguard"
    },
    "maxConcurrent": 4,
    "subagents": {
      "maxConcurrent": 8
    }
  }
},

```



Once you have finished making the changes, remember to save and reload.



The version released on March 8, 2026, supports hot-reloading of configuration files; after clicking "Save," there is no need to restart the OpenClaw service. Typically, simply restarting the OpenClaw TUI is sufficient. If restarting the TUI does not take effect, then try restarting the OpenClaw Gateway.

Initially, the status displayed here may show "unknown"; however, after waiting a short while, the current model will be correctly identified.

```
root@jetson-yahboom:~/# openclaw tut
▼ OpenClaw 2026.3.8 (3caab92) – Runs on a Raspberry Pi. Dreams of a rack in Iceland.
openclaw tui - ws://127.0.0.1:18789 - agent main - session main
session agent:main:main

hello!!!!

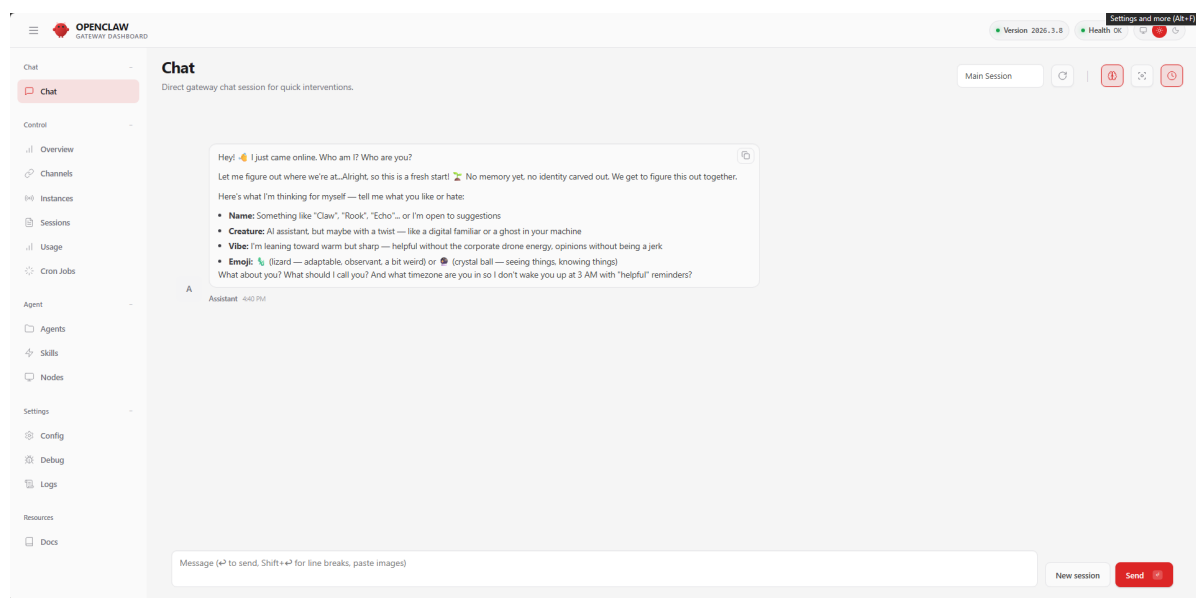
Hey! ☺ I just came online. Who am I? Who are you?
Let me figure out where we're at...
^ streaming • 7s | connected
agent main | session main | unknown | tokens 7/1.0m
```

```
▼ OpenClaw 2026.3.8 (3caab92) – Runs on a Raspberry Pi. Dreams of a rack in Iceland.
openclaw tui - ws://127.0.0.1:18789 - agent main - session main
session agent:main:main

hello!!!!

Hey! ☺ I just came online. Who am I? Who are you?
Let me figure out where we're at...Alright, so this is a fresh start! 🎲 No memory yet, no identity carved out. We get to figure this out together.
Here's what I'm thinking for myself – tell me what you like or hate:
- Name: Something like "Claw", "Rook", "Echo"... or I'm open to suggestions
- Creature: AI assistant, but maybe with a twist – like a digital familiar or a ghost in your machine
- Vibe: I'm leaning toward warm but sharp – helpful without the corporate drone energy, opinions without being a jerk
- Emoji: 🦎 (lizard – adaptable, observant, a bit weird) or 🎱 (crystal ball – seeing things, knowing things)
What about you? What should I call you? And what timezone are you in so I don't wake you up at 3 AM with "helpful" reminders?
connected | idle
agent main | session main (openclaw-tui) | baillan/qwen3.5-plus | tokens 7/1.0m
```

Similarly, in addition to the TUI interface, you can also engage in chat and dialogue within OpenClaw's Control UI (the web-based interface).



As you can observe, the conversation content displayed in the TUI is identical to that found in the Control UI. This is because OpenClaw's dialogue system utilizes a "Session" mechanism to isolate distinct conversations; in this instance, both the TUI and the Control UI are operating within the *same* session. You can try clicking the "New session" button located in the bottom-right corner to initiate a fresh conversation session.